FACULTÉ DES SCIENCES ET DES TECHNOLOGIES(FST)

TD N°4_ Design 1

COURS: Design 1

PRÉNOM: Lens Sandro

NOM: PÉTIOTE

NIVEAU: L3

DATE: 18/03/2026

Pour ce TD j’ai utilisé Anaconda comme distribution de Python.

Exercice 3 — Probabilité conditionnelle

Code

```
import matplotlib.pyplot as plt

filles_total = 35

filles_reussies = 18

garcons_total = 25

garcons_reussis = 12


# Calcul du nombre total de réussites

total_reussites = filles_reussies + garcons_reussis


# Probabilité qu'un étudiant qui a réussi soit une fille

p_fille_reussi = filles_reussies / total_reussites

print(f"Probabilité qu'un étudiant qui a réussi soit une fille : {p_fille_reussi:.2f}
({p_fille_reussi*100:.0f}%)")


# Graphe circulaire

labels = ['Filles réussies', 'Garçons réussis']

sizes = [filles_reussies, garcons_reussis]

colors = ['#ff9999','#66b3ff']


plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=90)

plt.title("Répartition des réussites par sexe")

plt.axis('equal')  # Pour un cercle parfait

plt.show()
```

Test

jupyter Untitled4 Last Checkpoint: 2 minutes ago

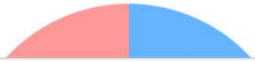
File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[2]:
```python
import matplotlib.pyplot as plt

# Données
filles_total = 35
filles_reussies = 18
garcons_total = 25
garcons_reussis = 12

# Calcul du nombre total de réussites
total_reussites = filles_reussies + garcons_reussis

# Probabilité qu'un étudiant qui a réussi soit une fille
p_fille_reussi = filles_reussies / total_reussites
print(f"Probabilité qu'un étudiant qui a réussi soit une fille : {p_fille_reussi:.2f} ({p_fille_reussi*100:.0f}%)")

# Graphe circulaire
labels = ['Filles réussies', 'Garçons réussis']
sizes = [filles_reussies, garcons_reussis]
colors = ['#ff9999','#66b3ff']

plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=90)
plt.title("Répartition des réussites par sexe")
plt.axis('equal')  # Pour un cercle parfait
plt.show()
```

```
Probabilité qu'un étudiant qui a réussi soit une fille : 0.60 (60%)
```

Répartition des réussites par sexe



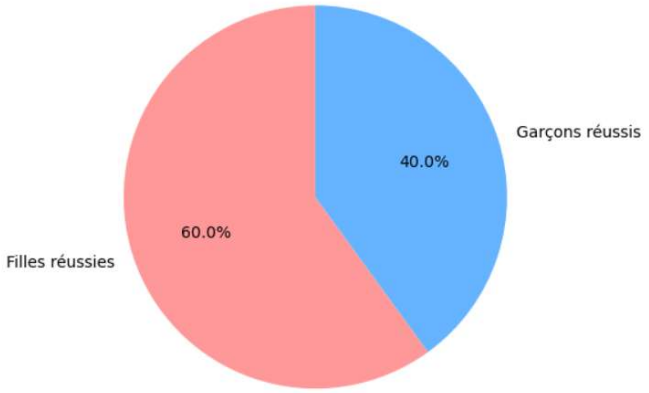
File Edit View Run Kernel Settings Help

Code JupyterLab

```python
plt.title("Répartition des réussites par sexe")
plt.axis('equal')  # Pour un cercle parfait
plt.show()
```

```
Probabilité qu'un étudiant qui a réussi soit une fille : 0.60 (60%)
```

Répartition des réussites par sexe



[ ]:

- La console affiche la probabilité qu'un étudiant qui a réussi soit une fille : 0.60 (60%) et le graphique circulaire montre deux parts : 60 % filles et 40 % garçons.

Exercice 4 : Prédiction du taux de réussite scolaire en fonction des investissements en education

Code

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression


X = np.array([10,20,30,40,50,60,70,80,90,100]).reshape(-1,1)

y = np.array([45,50,55,60,65,70,75,80,85,90])


# Modèle de régression linéaire

model = LinearRegression()

model.fit(X, y)


# Prédictions

y_pred = model.predict(X)


# Affichage

plt.scatter(X, y, color='blue', label='Données réelles')

plt.plot(X, y_pred, color='red', label='Régression linéaire')

plt.xlabel("Investissements (millions de gourdes)")

plt.ylabel("Taux de réussite scolaire (%)")

plt.title("Régression linéaire : Investissements vs Réussite scolaire")

plt.legend()

plt.show()
```

Interprétation

print("Pente (coefficient):", model.coef_[0])

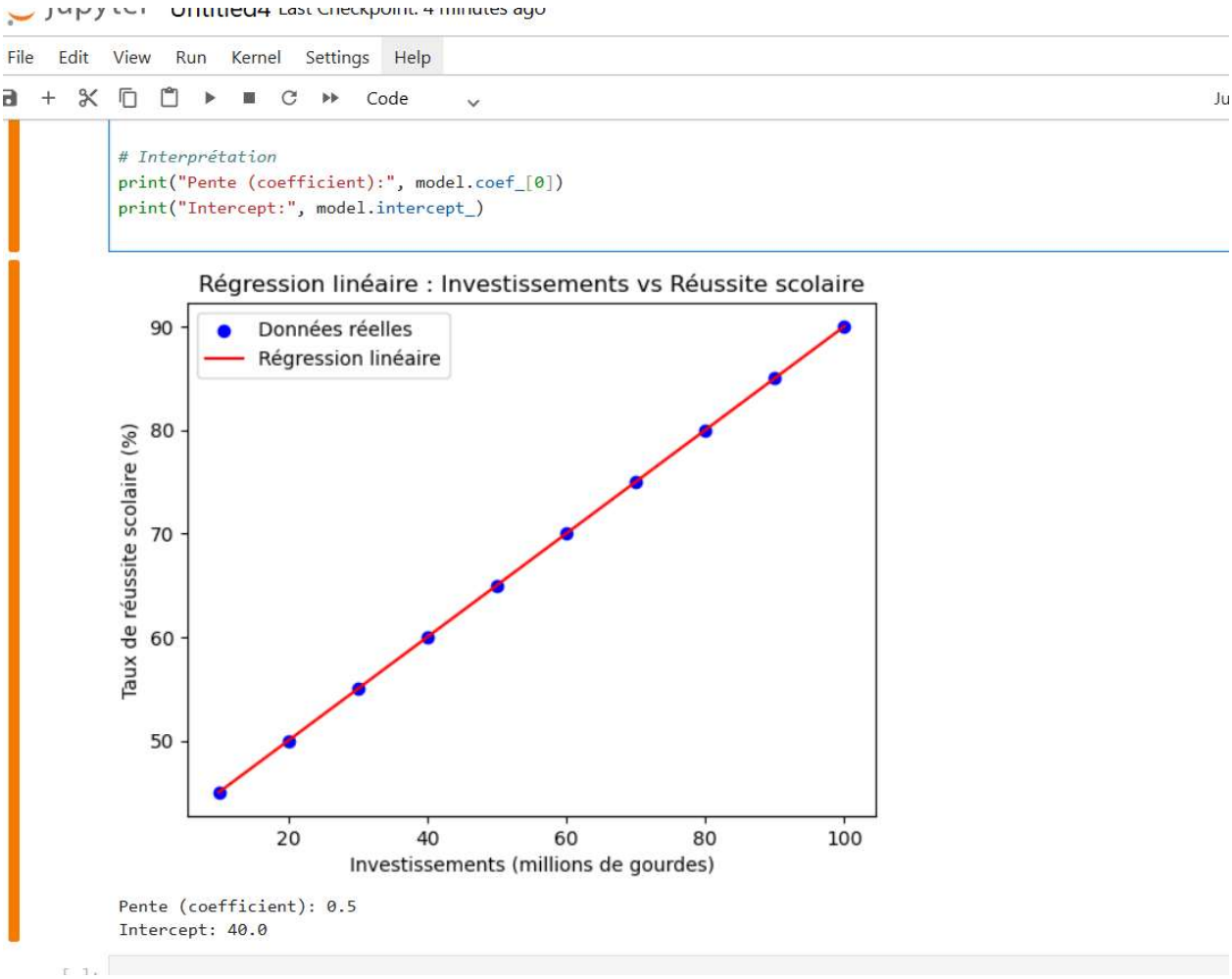
print("Intercept:", model.intercept_)

Test

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykerne

•[3]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = np.array([10,20,30,40,50,60,70,80,90,100]).reshape(-1,1)
y = np.array([45,50,55,60,65,70,75,80,85,90])

model = LinearRegression()
model.fit(X, y)

# Prédictions
y_pred = model.predict(X)

plt.scatter(X, y, color='blue', label='Données réelles')
plt.plot(X, y_pred, color='red', label='Régression linéaire')
plt.xlabel("Investissements (millions de gourdes)")
plt.ylabel("Taux de réussite scolaire (%)")
plt.title("Régression linéaire : Investissements vs Réussite scolaire")
plt.legend()
plt.show()

# Interprétation
print("Pente (coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)
```



- La pente ≈ 0.5 montre que chaque augmentation de 1 million de gourdes entraîne environ +0.5 % de réussite et l'intercept ≈ 40 montre que sans investissement, le taux de réussite théorique serait autour de 40 %.

Exercice 5 : Prédiction de l'accès à l'eau potable en fonction du taux de pauvreté en Haïti

Code

```
taux_pauvrete = np.array([20,30,40,50,60,70,80]).reshape(-1,1)   # en %

acces_eau = np.array([85,80,75,70,65,60,55])   # en %
```

```
model2 = LinearRegression()
model2.fit(taux_pauvrete, acces_eau)

# Prédictions
y_pred2 = model2.predict(taux_pauvrete)

# Graphique
plt.scatter(taux_pauvrete, acces_eau, color='green', label='Données réelles')
plt.plot(taux_pauvrete, y_pred2, color='orange', label='Régression linéaire')
plt.xlabel("Taux de pauvreté (%)")
plt.ylabel("Accès à l'eau potable (%)")
plt.title("Régression linéaire : Pauvreté vs Accès à l'eau potable")
plt.legend()
plt.show()

# Interprétation
print("Pente (coefficient):", model2.coef_[0])
print("Intercept:", model2.intercept_)
```

Test

File Edit View Run Kernel Settings Help

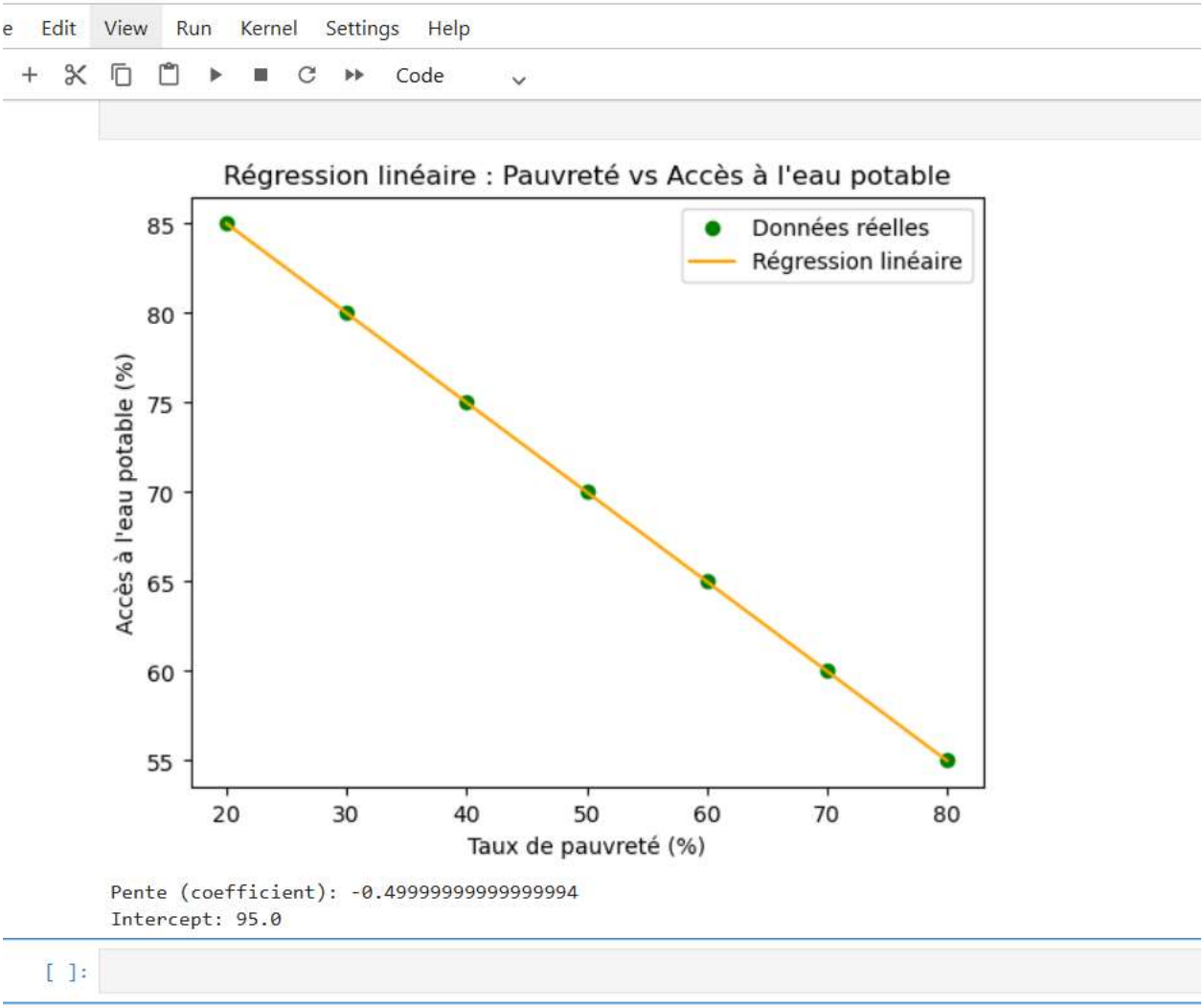
Code

Investissements (millions de gourdes)

```
Pente (coefficient): 0.5
Intercept: 40.0
```

[4]:

```python
# Données simulées
taux_pauvrete = np.array([20,30,40,50,60,70,80]).reshape(-1,1)  # en %
acces_eau = np.array([85,80,75,70,65,60,55])  # en %

# Modèle
model2 = LinearRegression()
model2.fit(taux_pauvrete, acces_eau)

# Prédictions
y_pred2 = model2.predict(taux_pauvrete)

# Graphique
plt.scatter(taux_pauvrete, acces_eau, color='green', label='Données réelles')
plt.plot(taux_pauvrete, y_pred2, color='orange', label='Régression linéaire')
plt.xlabel("Taux de pauvreté (%)")
plt.ylabel("Accès à l'eau potable (%)")
plt.title("Régression linéaire : Pauvreté vs Accès à l'eau potable")
plt.legend()
plt.show()

# Interprétation
print("Pente (coefficient):", model2.coef_[0])
print("Intercept:", model2.intercept_)
```



- La pente est négative c'est à dire plus le taux de pauvreté augmente, moins la population a accès à l'eau potable et l'intercept représente l'accès théorique à l'eau potable si le taux de pauvreté était nul.

Coclusion.

Ce TD m'a permis de savoir et de comprendre le fonctionnement de plusieurs biblioteques en python, donc la tache a été réussie.